FAKULTA MECHATRONIKY,
INFORMATIKY A MEZIOBOROVÝCH
STUDIÍ TUL

Diplomová práce

Architektura systému pohledové inspekce kvality

Studijní program: N0613A140028 – Informační technologie
Studijní obor: N0613A140028AI – Aplikovaná informatika
Autor práce: **Bc. Kevin Daněk**
Vedoucí práce: Ing. Igor Kopetschke

Liberec 2025

Tento list nahraďte originálem zadání.

PROHLÁŠENÍ

Prohlašuji, že svou diplomovou práci jsem vypracoval samostatně jako původní dílo s použitím uvedené literatury a na základě konzultací s vedoucím mé diplomové práce a konzultantem.
Jsem si vědom toho, že na mou diplomovou práci se plně vztahuje zákon č. 121/2000 Sb., o právu autorském, zejména § 60 – školní dílo.
Beru na vědomí, že Technická univerzita v Liberci nezasahuje do mých autorských práv užitím mé diplomové práce pro vnitřní potřebu Technické univerzity v Liberci.
Užiji-li diplomovou práci nebo poskytnu-li licenci k jejímu využití, jsem si vědom povinnosti informovat o této skutečnosti Technickou univerzitu v Liberci; v tomto případě má Technická univerzita v Liberci právo ode mne požadovat úhradu nákladů, které vynaložila na vytvoření díla, až do jejich skutečné výše.
Současně čestně prohlašuji, že text elektronické podoby práce vložený do IS STAG se shoduje s textem tištěné podoby práce.
Beru na vědomí, že má diplomová práce bude zveřejněna Technickou univerzitou v Liberci v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů.
Jsem si vědom následků, které podle zákona o vysokých školách mohou vyplývat z porušení tohoto prohlášení.

3. 3. 2026 Bc. Kevin Daněk

ARCHITEKTURA SYSTÉMU POHLEDOVÉ INSPEKCE KVALITY

ABSTRAKT

Tato zpráva popisuje třídu `tulthesis` pro sazbu absolventských prací Technické univerzity v Liberci pomocí typografického systému LaTeX.

Klíčová slova: LaTeX, třída, TUL

ABSTRACT

This report describes the `tulthesis` package for Technical university of Liberec thesis typesetting using the LaTeX typographic system.

Keywords: LaTeX, class, TUL

PODĚKOVÁNÍ

Dál bych chtěl poděkovat i zbytku akademické obce, se kterou jsem měl co dočinění. I když se s některými jejími členy, ozvláště pak z nadřízených pozic, neshodnu ani na počasí, tak díky každému jsem získal novou zkušenost. Někdy byla pozitivní, a někdy zase i negativní - to ale k životu patří.
A na závěr musím poděkovat svojí drahé polovičce, Simče Daňkové, která mě podporovala po celou dobu psaní této práce. Na začátku této práce byla mou přítelkyní, a nyní, na jejím konci, je již mojí manželkou. Díky ní si budu tuto práci pamatovat nejenom jako svůj *magnum opus*, ale jako životní milník, kterým se změnil skoro každý aspekt mého života. Pokud Vás zajímají regionální dějiny a příběhy z časů minulých, doporučuji si přečíst její bakalářskou práci, kterou tvořila souběžně s tou mojí, a zabývá se tématem sebevražd po druhé světové válce na Liberecku a obzvláště v Jablonci nad Nisou a jeho blízkém okolí.

OBSAH

Obsah 6
Seznam zkratek 8

1 Úvod **9**

2 Osvědčené techniky softwarového inženýrství **10**
2.1 Technický dluh 11
2.2 Clean Code 13
2.2.1 Identifikátory 13
2.2.2 Funkce 14
2.3 Návrhové vzory 16
2.4 Testování 16
2.5 Verzování 16
2.6 Postupná integrace a postupné nasazení 16
2.7 Monitorování nasazeného systému 16
2.7.1 Logy 16
2.7.2 Metriky 17

3 Architektura nasazeného řešení **18**
3.1 Analýza stávajícího řešení 18
3.1.1 Diagram systémového kontextu 18
3.1.2 Diagram kontejnerů 19
3.1.3 Diagram komponent 19
3.1.4 Komunikace mezi kontejnery 19
3.1.5 Absence návrhových vzorů 19
3.1.6 Absence architektury 19
3.1.7 Absence zotavení z výjimky 19
3.1.8 Absence procesu sestavení a nasazení 19
3.1.9 Absence použitelné uživatelské dokumentace 19

4 Mitigace chyb v archiktuře **20**
4.1 Definice softwarových požadavků 20
4.1.1 Přehled produktu 21
4.1.2 Přehled produktu 21
4.1.3 Přehled ověřovacích metod 21
4.2 Mitigace chyb v archiktuře 21

4.2.1 Rozbití monolitu 21
4.2.2 Přidání definic funkcí a životních cyklů v řešení 22
4.2.3 Přidání brokeru pro systémové zprávy a události 22
4.2.4 Normalizování schématu relační databáze 22
4.3 Mitigace chyb v návrhu databáze 22
4.3.1 Přepsání API rozhraní do kompilovaného jazyka 23
4.3.2 Využití dokumentové databáze pro práci s JSON soubory 26
4.3.3 Předělání systému projektů 26
4.3.4 Přidání systému specifikací pro definici inspekce 26
4.3.5 Přidání systému pro správu toolů 26
4.3.6 Přidání důkladné autentizace a autorizace 27
4.3.7 Přidání procesu sestavení řešení 27
4.3.8 Předělání procesu nasazení 27
4.3.9 Přidání observability a monitoringu 27
4.3.10 Přidání licenčního systému 27

5 Závěr **30**

Přehled příkazů, prostředí a voleb 31

SEZNAM ZKRATEK

CD	Continuous Deployment (Postupné nasazení)
CI	Continuous Integration (Postupná integrace)
DevOps	Development Operations
GPG	GNU Privacy Guard
JSON	Javascript Object Notation
JWT	JSON Web Tokens
SecOps	Security Operations
SSH	Secure Shell
TPM	Trusted Platform Module

1 ÚVOD

Každý, jakkoliv velký, systém musí někdo navrhnout. S rostoucí složitostí systému zároveň roste i složitost jeho správného návrhu. Ve své předchozí práci jsem se věnoval rapidnímu vývoji algoritmů strojového vidění, jehož výsledkem byla nová funkční komponenta pro trénování a inferenci neuronových sítí a jiných algoritmů strojového vidění. (Daněk, 2024)

Již tehdy byla samotná implementace omezena návrhem existujícího systému. Upřednostňování rychlého dodání nových funkcionalit na úkor průběžné údržby kódu vyústilo v enormní množství technologického dluhu, který se za sedm let ve zdrojovém kódu nastřádal. Tato práce navazuje na mou předchozí práci a dívá se na ní ze širší perspektivy. Cílem je odklonit se od vývoje nových funkcí a obrátit pozornost právě k zanedbané architektuře celého řešení. Rozsah analýzy jsem stanovil nad rámec softwarového návrhu a věnuji se rovněž architektuře infrastruktury nasazeného řešení.

2 OSVĚDČENÉ TECHNIKY SOFTWAROVÉHO INŽENÝRSTVÍ

Programátor vybavený klávesnicí má téměř neomezené možnosti realizace svých nápadů, podobně jako malíř se štětcem. Je však pozoruhodné, že na rozdíl od nekontrolovaného tvůrčího procesu malíři často své možnosti úmyslně limitují. Právě dodržením určitých kompozičních pravidel totiž docílí toho, že obraz není pouze chaotickou směsí čar a barev, ale srozumitelnou myšlenkou, kterou divák dokáže instinktivně rozpoznat.

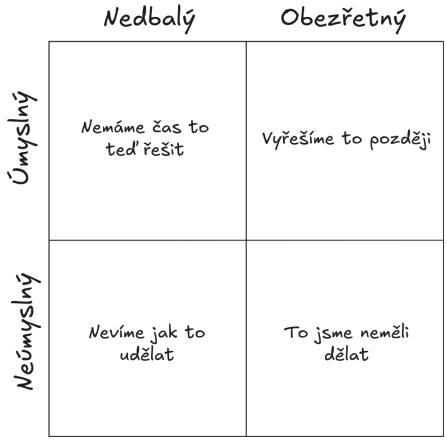
Principy vývoje softwaru se od této úvahy příliš neliší. Pokud se při vývoji zavážeme k dodržování stanovených pravidel a konvencí, umožníme ostatním vývojářům snadněji pochopit záměr našeho kódu. Této oblasti se právě věnuje **softwarové inženýrství**, které se zabývá návrhem kódu, který je udržitelný, čitelný, spravovatelný a testovatelný (Michigan Technological University, 2025). Z předvídatelného a čitelného kódu profituje i samotný autor, který se tak při návratu k vlastní práci po delším časovém odstupu nemusí věnovat náročnému dekódování původního záměru.

Softwarové inženýrství je velmi rozsáhlý obor a není v možnostech této práce jej pojmout v jeho úplnosti. Teoretický rámec se proto omezuje na vybraná témata, která jsou klíčová pro pochopení praktické části. Kapitolu otevírá pojednání o technickém dluhu, jeho příčinách a metodách jeho eliminace. Na tento koncept navazuje pasáž věnovaná principům udržitelného a testovatelného kódu, které tvoří nezbytný základ pro následné návrhové vzory. Ty využívají zmíněné principy a transformují je do osvědčených řešení běžných architektonických problémů.

Další část práce se věnuje metodice efektivního testování a principům kontinuální integrace a doručování (CI/CD). Teoretický blok uzavírá kapitola o monitoringu nasazeného řešení, který poskytuje nezbytný vhled do chování systému v produkčním prostředí, kde vývojář zpravidla nemá k dispozici stejné diagnostické nástroje jako v lokálním vývojovém prostředí.

2.1 TECHNICKÝ DLUH

Technický dluh vzniká v situacích, kdy dojde k nedodržení doporučených postupů nebo k volbě suboptimálního návrhu s cílem urychlit dokončení vývoje. Krátkodobě ušetřený čas se v budoucnu projevuje jako dlouhodobá zátěž, zejména v momentě, kdy je zapotřebí systém dále rozšiřovat či upravovat. Akceptace technického dluhu nemusí být nutně negativním krokem, je však nezbytné s ním nakládat opatrně. Čas, který si v daný moment „vypůjčíme“ pro urychlení vývoje, musí být v budoucnu splacen. Je pak na zodpovědnosti vývojářů a managementu, zda budou tento dluh umořovat průběžně, nebo vyčkají, dokud jeho výše neznemožní další produktivní práci.



Obrázek 2.1: Kvadranty technického dluhu (Team Asana, 2025)

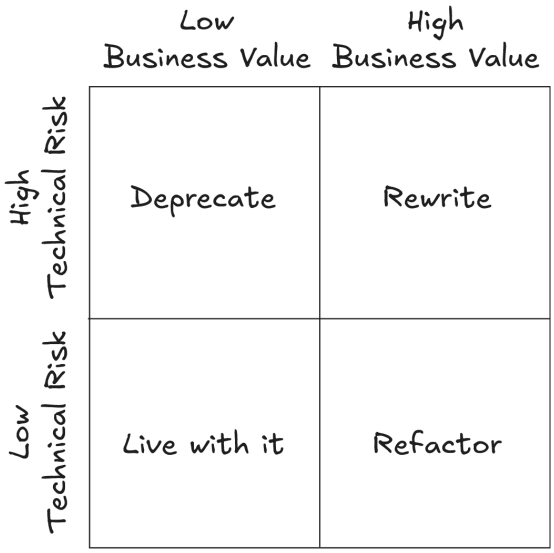
Martin Fowler definuje tzv. kvadrant technického dluhu, ve kterém jej rozděluje do čtyř kategorií na základě dvou os: zda byl dluh vytvořen úmyslně a zda byl vývojář v daný okamžik obezřetný. Úmyslný dluh vzniká vědomým rozhodnutím upřednostnit rychlost dodání před kvalitou. Neúmyslný dluh je naopak důsledkem nezkušenosti nebo nedostatku znalostí v době návrhu. Rozlišení na dluh obezřetný a nedbalý pak odráží míru profesionality a zhodnocení rizik – zatímco obezřetný dluh je přijat s vědomím budoucích nákladů na jeho splacení, nedbalý dluh vzniká ignorováním základních principů softwarového inženýrství (Fowler, 2009).

Identifikace technického dluhu v rámci softwarového projektu může probíhat několika způsoby. Prvotním indikátorem je obvykle dynamika samotného vývoje. Pokud se rychlost vývoje (tzv. velocity) snižuje a čas potřebný k vydání nové verze neúměrně roste, bývá jednou z příčin právě nutnost vývojářů neustále překonávat bariéry v podobě zanedbaného či nepřehledného kódu. Subjektivní, avšak kriticky důležité hodnocení poskytují i samotní vývojáři, kteří jsou schopni přesně identifikovat konkrétní úseky systému, jež je při práci nejvíce zpomalují (Team Asana, 2025).

Objektivnější pohled nabízejí nástroje pro statickou analýzu kódu, které dokáží automatizovaně detekovat duplicity (tzv. code smells) nebo antipatterny naznačující architektonické nedostatky. Významnou metriku představuje rovněž code churn – ukazatel četnosti změn v konkrétních částech kódu. Vysoká míra fluktuace změn v určitém modulu často signalizuje jeho nevhodný návrh, nestabilitu nebo přílišnou provázanost s ostatními částmi systému, což jej činí náchylným k chybám a náročným na údržbu (Team Asana, 2025).

Příčiny vzniku technického dluhu mohou být jak technické tak procesní. V technické rovině jde především o absenci smysluplné dokumentace, která by popisovala architekturu a chování systému, a o nízké pokrytí kódu testy. Automatizované testy totiž fungují také jako popis očekávaného chování systému. V rovině procesní pak dluh vzniká jako je nátlak vedení na rychlost dodání, nebo v důsledku soustavného odkládání nezbytného refactoringu na neurčito (Shvets, 2018).

Strategie řešení technického dluhu závisí na analýze rozsahu, rizik a dostupného rozpočtu. U méně kritických systémů, jejichž případný výpadek nepředstavuje vysoké riziko, nemusí být eliminace dluhu prioritou. Naopak u kritických komponent s vysokou mírou poruchovosti a nízkou udržitelností je nutné zvážit, zda je efektivnější refaktorizace, nebo kompletní přepsání systému.



Obrázek 2.2: Kvadranty řešení technického dluhu (ForrestKnight, 2025)

2.2 CLEAN CODE

Clean Code představuje soubor pravidel a doporučení, jejichž primárním cílem je snížit kognitivní zátěž při čtení a údržbě zdrojového kódu (Martin; Coplien, 2009). Na clean code je pohlíženo jako na myšlenkový pochod, který má za cíl sjednotit komunikaci záměrů a zjednodušit údržbu či rozšíření kódu po určitém časovém odstupu. Jak píše i samotný Robert Martin ve své knize[1], jednotlivé zásady nejsou absolutní pravdou a míra jejich aplikace může být předmětem diskuse v týmu. V následujících podkapitolách jsou rozebrány vybrané zásady, které mají podle mých zkušeností největší vliv na čitelnost a dlouhodobou udržitelnost systému.

Je důležité zdůraznit, že cílem Clean Code není přesné dodržování arbitrárních pravidel. Smyslem je zmírnit negativní dopady neudržitelného kódu – často psaného pod časovým tlakem – na budoucí produktivitu celého týmu. Žádoucím výsledkem je udržení konstantního tempa vývoje v čase a eliminace potřeby rozsáhlých a rizikových přepisů, či celkové refaktorizace projektu.

2.2.1 Identifikátory

Vývoj softwaru vyžaduje pojmenovávání široké škály konstrukcí – od proměnných a funkcí až po třídy, rozhraní a specifické jazykové struktury. Zásadním principem je, aby každý název nesl jasný význam a záměr. Název identifikátoru by měl zodpovídat otázky jako: „Proč tato entita existuje?“, „Co dělá?“ a „Jakým způsobem se používá?“. Například jednopísmenný název proměnné neposkytuje žádnou informaci o jejím účelu. Pokud však řešíme výpočet délky přepony pravoúhlého trojúhelníku, název `delkaPreponyCm` jasně definuje obsah i jednotky dané hodnoty (Martin; Coplien, 2009).

Ke srozumitelnosti názvu patří i zdánlivě jasné pravidlo. Název nesmí lhát. Nenazývejme věc seznamem, pokud to skutečně není seznam. Zatímco od seznamu programátor očekává možnost indexace přirozenými čísly, u slovníku je tento předpoklad chybný a kód se stává neintuitivním. S tím souvisí i nebezpečí příliš podobných názvů. Ve vývojových prostředích, která nabízejí našeptávání kódu nebo automatické doplňování pomocí AI asistentů, může minimální rozdíl v názvech vést k fatálním, těžko odhalitelným, chybám (Martin; Coplien, 2009).

Jednotlivé identifikátory by se měly lišit natolik, aby bylo možné sledovat roli, kterou v systému plní. Pojmenování argumentů funkce stejným slovem s numerickým sufixem (např. `data1`, `data2`) neposkytuje žádnou informaci o jejich sémantickém významu. Kromě číslovek se lze často setkat s přístupem přidání afixů jako `manager` nebo `data`. Pokud názvy od sebe odlišíme akorát těmito afixy, ztrácí se smysl daných proměnných.

[1] MARTIN, Robert C.; COPLIEN, James O., 2009. *Clean code: a handbook of agile software craftsmanship*. Prentice Hall. ISBN 9780132350884.

Mnohdy totiž vývojář použije název s afixem jenom proto, že identifikátor již v kontextu existuje a nenapadá ho lepší název. Pokud dva identifikátory nesou různý význam, musí být i jejich názvy různé (Martin; Coplien, 2009).

Identifikátory by měli být hledatelné a vyslovitelné. Pokud má vývojový tým o kódu efektivně diskutovat, musí být názvy použitelné v běžné řeči bez nutnosti hláskovat nesrozumitelné zkratky. Vyhledatelnost pak úzce souvisí s unikátností názvu v rámci kontextu. Vyhneme se tím dlouhému přemýšlení nad tím, co daná zkratka znamená, a můžeme se místo toho soustředit na podstatu věci. V této souvislosti je rovněž žádoucí vyhýbat se tzv. „magickým konstantám“ v řídicích strukturách a místo nich definovat pojmenované konstanty na odpovídající úrovni abstrakce (Martin; Coplien, 2009).

Při snaze o odlišení je však nutné vyvarovat se kódování technických informací (např. datového typu) přímo do názvu, což je relikt dřívějších dob (tzv. maďarská notace). V éře statické analýzy není nutné, aby název proměnné obsahoval informaci o tom, že jde o `int`, nebo aby název rozhraní začínal písmenem `I`. Kolize názvu rozhraní a třídy (např. `Camera`) by neměla být řešena prefixem `ICamera`, ale raději sémantickým upřesněním konkrétní implementace – například `CameraService` nebo `PhysicalCamera`. Takový přístup do názvu vkládá informaci o roli či principu fungování, nikoliv pouze o jazykové konstrukci (Martin; Coplien, 2009).

2.2.2 Funkce

Funkce a procedury jsou většinou první úrovní abstrakce a metodou pro členění kódu. *Clean Code* proto shrnuje pravidla pro návrh funkcí do tří principů: funkce by měla být krátká, měla by vykonávat právě jednu činnost a měla by svoje působení omezit na kontext funkce (nesmí mít *vedlejší účinky*) (Martin; Coplien, 2009).

Požadavek na minimalizaci délky funkce je především varováním před přílišnou hloubkou zanoření. Vysoká míra zanoření v řídicích strukturách obvykle indikuje nekonzistentní úroveň abstrakce a naznačuje, že by konkrétní bloky kódu měly být refaktorovány do samostatných metod. Jak poznamenal Linus Torvalds: *„… if you need more than 3 levels of indentation, you're screwed anyway, and should fix your program.“*

Princip jediné činnosti na úrovni funkce znamená, že by v rámci jejího těla měla být zachována jednotná úroveň abstrakce. Funkce by měla buď vykonávat příkaz, nebo odpovídat na dotaz, ale nikdy obojí zároveň (princip *Command Query Separation*). Vysokoúrovňová logika by neměla být zatěžována nízkoúrovňovými výpočty; ty by měly být delegovány na specializované podfunkce. Tento hierarchický přístup lze přirovnat k fungování firmy: ředitel se věnuje strategickému řízení a logistické procesy deleguje na vedoucí skladů, kteří dále řídí konkrétní naskladnění zboží (Martin; Coplien, 2009).

Pro udržení konzistentní abstrakce je doporučeno dodržovat pravidlo sestupného čtení (The Stepdown Rule). Program by měl být čitelný jako posloupnost odstavců, kde každý rozvíjí detaily předchozího, uvozený pomyslnou předponou „k“ (v anglickém originále *to*):

- K vyřízení objednávky je třeba zabalit zboží, odečíst jej ze skladu a vytisknout štítek.
- K zabalení zboží je třeba položku vyhledat a zvolit vhodný obal.
- K odečtení ze skladu je třeba identifikovat pozici zboží v registru.
- K vytisknutí štítek je třeba určit přepravce a vygenerovat tisková data.

Komplexitu funkce však neurčuje pouze její tělo, ale také počet jejích argumentů. Ideální funkce je niladická (bez argumentů). Běžné jsou funkce monadické (jeden argument) a dyadické (dva argumenty). Přítomnost triadické (tři argumenty) nebo dokonce polyadické funkce (čtyři a více) je vnímána jako varovný signál nevhodného návrhu (Martin; Coplien, 2009).

S rostoucím počtem parametrů roste také počet testovacích případů, pokud předpokládáme čtyři parametry, které jsou pravdivostní hodnoty, jedná se o $2^4 = 16$ kombinací, které je potřeba otestovat. Podobně můžeme postupovat i u spojitých proměnných, kde vybíráme zástupce z jednotlivých tříd ekvivalence. Menší počet argumentů nejenom zjednodušejí použití funkce, ale i její testování.

S rostoucím počtem parametrů roste i složitost testování. Uvažujeme-li funkci se čtyřmi parametry typu *boolean*, vzniká celkem $2^4 = 16$ kombinací vstupních stavů, které je nutné validovat. U spojitých proměnných je situace ještě složitější a vyžaduje pečlivý výběr zástupců z tříd ekvivalence. Rostoucí počet argumentů také zavádí problém, kdy mezi nimi chybí přirozené pořadí. Hrozí tak riziko záměny, pokud nás vlastní intuice nenavede správně[2].

Pokud funkce vyžaduje velký počet parametrů, je vhodné zvážit jejich seskupení do logických datových struktur nebo objektů. *Clean Code* rovněž důrazně varuje před používáním tzv. příznakových argumentů (flag arguments). Pokud funkce přijímá *boolean*, který větví tok funkce, z definice vykonává více než jednu věc a měla by být rozdělena do dvou samostatných funkcí.

[2]Existují situace, kdy mezi parametry je přirozené uspořádání, například souřadnice v kartézské soustavě, kdy jsou souřadnice tvořeny uspořádanou dvojicí (x, y)

2.3 NÁVRHOVÉ VZORY

2.4 TESTOVÁNÍ

2.5 VERZOVÁNÍ

2.6 POSTUPNÁ INTEGRACE A POSTUPNÉ NASAZENÍ

2.7 MONITOROVÁNÍ NASAZENÉHO SYSTÉMU

Kritickou součástí vývoje softwaru je jeho testování a ladění (debugging). Testy, například jednotkové nebo integrační, simulují chování systému za různých podmínek a následné ladění pomocí debuggerů odhaluje jeho vnitřní stav instrukce po instrukci. V praxi ovšem není možné odhalit všechny chyby a dojde na situace, kdy se v nasazeném řešení chyba objeví. Při lokálním vývoji by se program odkrokoval debuggerem a zjistili se okolnosti, které chybu způsobily.

V produkčním nasazení ovšem luxus odkrokování programu zpravidla nemáme, ať už kvůli chybějícím pomocným strukturám (například *symbol table*) a nebo kvůli optimalizacím kompilátoru, který program převede do jazyka symbolických adres či byte kódu. Když tedy nelze pozorovat stav programu zevnitř, nezbývá nám nic jiného, než ho pozorovat zvenčí - a konkrétně přes jeho výstupy. Schopnost pozorovat chování systému na základě jeho výstupů nazýváme **observabilitou**.

Mnoho autorů a spolků, kteří se observabilitou zabývají, jí rozdělují na pilíře. Cílem těchto pilířů je zodpovědět na otázky: *Co se to děje?*, *Proč se to děje?* a *Kde se to děje?*. Každý pilíř tak doplňuje informace k těm dalším, a dohromady tvoří kompletní vhled do chování systému. (Elastic Observability Team, 2024) Tato práce ovšem bude na observabilitu nahlížet trochu jinak. Paradigma pilířů indikuje, že každý pilíř je disjunktním článkem, které, když se spojí dohromady, tvoří observabilitu.

2.7.1 Logy

Jedním z nejčastějších výstupů programu, který se používá pro ladění nasazeného řešení, jsou logy. Jedná se o textové záznamy, které nesou informaci o událostech v programu. Logování má své zásady a doporučení, jejichž dodržení či nedodržení ovlivňuje míru, kterou nám logy pomohou s diagnostikou problému. Ačkoliv je logování velmi užitečné, nedává nám

komplexní obraz o chování systému, protože se jedná jenom o jeden z možných výstupů programu. Program může vytvářet další telemetrická data, která mohou být následně dále zpracována.

2.7.2 Metriky

Metriky jsou kvantitativní veličiny, které měří konkrétní vlastnosti běžícího programu. Můžeme tak měřit například počet požadavků na konkrétní instanci či počet I/O operací. Metriky máme přímo měřené a odvozené, přičemž odvozené metriky můžeme zjišťovat z logů (například počet chyb v čase)

Je nutné podotknout, že metriky má smysl zaznamenávat dlouhodobě jakožto časové řady, abychom určili chování systému v čase. Je nutné tak metriky zaznamenávat na základě události, nebo pravidelného měření. Naopak zaznamenávat metriky vycházející ze statické analýzy smysl nedává, protože pokud bychom chtěli například analyzovat složitost programu pomocí Halsteadových vzorců, tak se tato metrika nemůže měnit v čase běhu programu. Naopak zatížení konkrétní instance programu nebo využití hardwarových prostředků proměnlivé čase běhu programu je.

3 ARCHITEKTURA NASAZENÉHO ŘEŠENÍ

3.1 ANALÝZA STÁVAJÍCÍHO ŘEŠENÍ

Cílem této kapitoly je provést podrobnou analýzu stávajícího řešení a identifikovat jeho případné nedostatky či zásadní chyby v návrhu, které mohou negativně ovlivnit běh, stabilitu a celkovou spolehlivost systému. K tomuto účelu byly využity tzv. C4 diagramy, což je metodika vizualizace softwarové architektury, která umožňuje nahlížet na systém z několika úrovní abstrakce – od celkového kontextu systému až po detailní zobrazení jednotlivých komponent. Tato strukturovaná forma modelování pomáhá nejen pochopit architekturu z pohledu různých cílových skupin (např. vývojáři, architekti, zadavatelé), ale také usnadňuje identifikaci slabých míst návrhu a jejich následnou optimalizaci.

3.1.1 Diagram systémového kontextu

Prvním diagramem je diagram systémového kontextu. Ten nám dovoluje modelovat systém jako střed diagramu, který je obklopen uživateli a dalšími systémy, se kterými interaguje. Cílem není tedy modelovat samotný systém, ale prostředí, ve kterém je nasazen. V kontextu CLOUCODE AITechDetect umožňuje modelovat, jací uživatelé se systémem interagují a jaké další systémy nasazené řešení potřebuje. V diagramu [doplň odkaz] je znázorněn diagram systémového kontextu pro AITechDetect. První věcí, kterou můžeme z diagramu zjistit, je fakt, že celé řešení není nadstavbou nad dalším řešením spravované jinou společností.

3.1.2 Diagram kontejnerů

3.1.3 Diagram komponent

3.1.4 Komunikace mezi kontejnery

3.1.5 Absence návrhových vzorů

3.1.6 Absence architektury

3.1.7 Absence zotavení z výjimky

3.1.8 Absence procesu sestavení a nasazení

3.1.9 Absence použitelné uživatelské dokumentace

4 MITIGACE CHYB V ARCHIKTUŘE

Po analýze celého systému je možné začít jednotlivé neduhy napravovat. Tato kapitola dokumentuje jednotlivé snahy o napravení nalezených problémů a slabin, pričemž zvláštní důraz je kladen na empirický přístup k implementaci nejlepšího řešení. Je tedy možné, že potenciální náprava problému se po měření ukáže jako ta, která působí více škody než užitku. Nelze takto ale měřit všechny problémové aspekty, a u takových případů je kladen důraz na *best-practices* vyjmenované v konkrétních CWE.

4.1 DEFINICE SOFTWAROVÝCH POŽADAVKŮ

Před samotným vývojem softwaru je velmi užitečné sepsat dokument, který jasně vymezí, co se od výsledného řešení očekává. Norma ISO 29148 takový dokument nazývá specifikace softwarových požadavků. Tato norma popisuje požadavky a související aspekty do velkého detailu, takže kompletní využití celé osnovy se v praxi obvykle nepoužívá. To ale neznamená, že by se měl tento přístup úplně zavrhnout – i stručná a základní verze specifikace pomáhá předejít chybným interpretacím zadání a následným nedorozuměním mezi vývojáři a zákazníky.

Specifikace by však neměla být jen seznamem funkcí. Důležitý je také kontext výsledného produktu. Dokument proto neslouží jen programátorům, ale i dalším zúčastněným stranám, jako jsou testeři nebo designeři. Ti potřebují znát prostředí, ve kterém bude software používán, a úkony, které má plnit. Typická specifikace softwarových požadavků se skládá ze tří hlavních částí:

- **Přehled produktu**, ve které se popíše produkt, jeho kontext, uživatelé a možná omezení.
- **Přehled požadavků**, která definuje požadavky na funkce, systémy, rozhraní, návrh či kvalitu.
- **Přehled ověřovacích metod**, která definuje způsoby ověření skutečnosti, že byl software vytvořen v souladu se zadáním.

4.1.1 Přehled produktu

Přehled o produktu, který je předmětem této práce, již částečně poskytla kapitola 3.1.1 s diagramem systémového kontextu. Tyto diagramy dobře zobrazují vztahy mezi produktem, jeho uživateli a externími systémy. Na druhou stranu se však příliš nevěnují samotnému produktu – chybí jim pohled dovnitř, tedy zaměření na jeho funkce a omezení.

Jádrem systému AITechDetect je proces inspekce. Ten probíhá ve třech krocích: pořízení fotografie, její vyhodnocení a následné uložení. Na tento základní proces navazují rozšiřující funkce, například vícepohledová inspekce, anotace snímků, archivace kontrol nebo správa specifikací a projektů.

4.1.2 Přehled produktu

4.1.3 Přehled ověřovacích metod

4.2 MITIGACE CHYB V ARCHIKTUŘE

4.2.1 Rozbití monolitu

Monolitický návrh řešení je pro začátek vývojového cyklu velmi výhodnou cestou. Umožňuje totiž velmi snadno komunikovat mezi částmi aplikace a přidávat nové funkcionality, které staví na těch předchozích. Výhodou je také to, že řešení se chová a pouští jako celek, který nepotřebuje k jeho běhu další závislosti.

Každý větší projekt se ovšem jednou dostane do fáze, kdy je třeba zvážit, jestli monolitický návrh projektu spíše neškodí. Neošetřená chyba v jedné části programu sebou stáhne celý program a části, které nijak s chybovým kódem nesouvisí, jsou kvůli pádu programu nedostupné. Pokud je monolit dostatečně velký, nese sebou i jistou míru komplexity, která stěžuje hledání chyb a jejich reprodukci. Navíc pokud je monolit orientovaný kolem určité technologie, např. kombinace Flask a Python, jsou do této technologie vývojáři uzamčeny, i když zrovna nemusí být ta nejideálnější, a v horším případě i aktivně nedoporučovaná.

Důvodů pro zachování monolitu i pro jeho rozbití je v případě CLOUDCODE AITechDetect mnoho, k jeho rozbití bylo nakonec přistoupeno s ohledem na budoucí implementaci CI/CD, monitoringu a způsobu nasazení pomocí kontejnerů, enginu Docker a Docker Compose pro orchestraci kontejnerů. Do kontejnerového diagramu tudíž přibudou nové kontejnery, které se převážně odštěpí od Flask Serveru.

- CCM-ML
- CCM-INSPEKCE

- CCM-FRONTEND
- CCM-BACKEND
- CCM-BROKER

Jedna věc je monolit rozbít, ale druhou je jednotlivé části opět spojit zpátky. K tomu má sloužit primárně kontejner CCM-BROKER, se kterým lze komunikovat pomocí HTTP nebo pomocí Websockets.

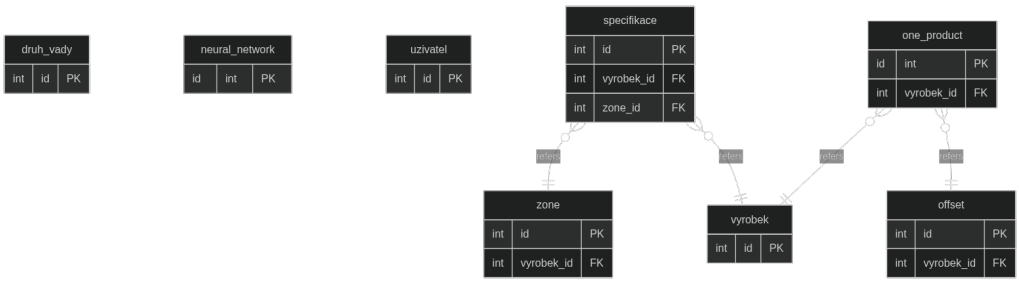
4.2.2 Přidání definic funkcí a životních cyklů v řešení

4.2.3 Přidání brokeru pro systémové zprávy a události

https://kafka.apache.org/documentation/

4.2.4 Normalizování schématu relační databáze

AITechDetect využívá relační databází SQLite pro uchovávání některých dat systému. V této kapitole je cílem podívat se na schéma této relační databáze a normalizovat ho na úroveň, kdy nebude docházet k aktualizačním anomáliím. Databáze obsahuje celkově 8 tabulek, které jsou ovšem svými atributy natolik rozsáhlé, že nebudou zakresleny do ER diagramu, ale podrobně popsány v následujícím textu.



Obrázek 4.1: Zkrácený ER Diagram původního schématu databáze

4.3 MITIGACE CHYB V NÁVRHU DATABÁZE

Normální formy

Návrh schématu

Při návrhu schématu relační databáze je třeba dbát na fakt, že databáze má modelovat vztahy mezi entitami z reálného světa. Je proto důležité mít výčet těchto entit a jaké mají mezi sebou závislosti. Mezi podstatné entity modelu AITechDetect patří:

- **Projekty**, neboli logická jednotka pro konkrétní druh výrobku a jeho souvisejících kontrol,
- **Vady**, které je možné nalézt na výrobku,
- **Kamery**, které jsou rozmístěné na kontrolní stanici,
- **Specifikace**, která určuje zásady pro vyhodnocení kontroly (kolik vad je ještě v pořádku),
- a **Kontroly**, které zaznamenávají výsledek jednotlivých kontrol inspekce.

Každá inspekce se odvíjí od projektů, které reprezentují jednotlivé druhy výrobků, které systém kontroluje.

Druhy vad

Tabulka `druh_vady` je minimálně v BCNF, a to čistě z toho důvodu, že obsahuje tři atributy - identifikační číslo vady, název a přepínač, jestli je vada považována na smazanou.

Vytvoření vlastního systému migrací

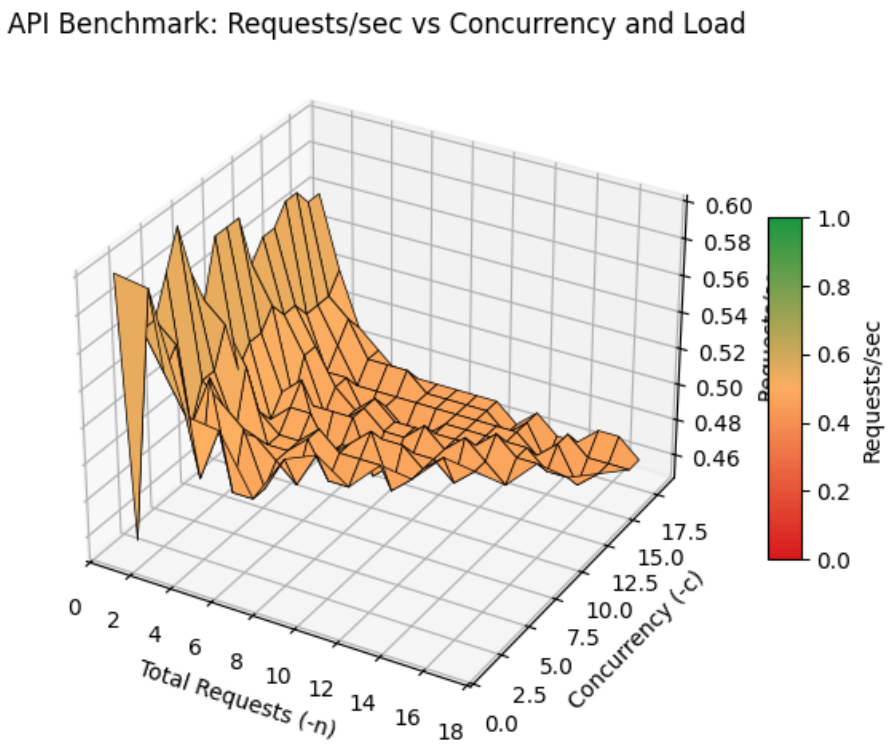
- `druh_vady`, která reprezentuje vadu na výrobku
- `uzivatel`, která reprezentuje přihlášeného uživatele v systému
- `neural_network`, která reprezentuje natrénovanou neuronovou síť
- `specifikace`, která definuje kolik vad a jakého druhu může být na jednom výrokbu.

4.3.1 Přepsání API rozhraní do kompilovaného jazyka

Webové API poskytující služby pro uživatelské rozhraní a další součásti systému AITechDetect je napsán v jazyce Python, který je interpretovaný. To sebou přináší několik obtíží a *overhead* v podobě interpreteru. Dalším problémem je fakt, že zdrojový kód aplikace je čitelný běžnému smrtelníkovi, což je v rámci politiky společnosti stav nežádoucí. Nabízela se tedy otázka, zdali by nebylo výhodné tuto část apliakce zkompilovat a jaký jazyk k tomu využít. Pro python existují kompilační nástroje, ovšem kompilace trvají dlouho a navíc přinášejí další zátěž do celého procesu.

Pro účely měření byl vytvořen testovací dataset o velikosti 1000 souborů JSON. Cílem úlohy bylo tyto data načíst a vrátit jako jeden velký JSON payload. Měření probíhalo pomocí nástroje Apache Benchmark, kde byl postupně zvyšován počet celkových požadavků a počet celkově souběžných požadavků. Měření byla provedena na stejném hardware se

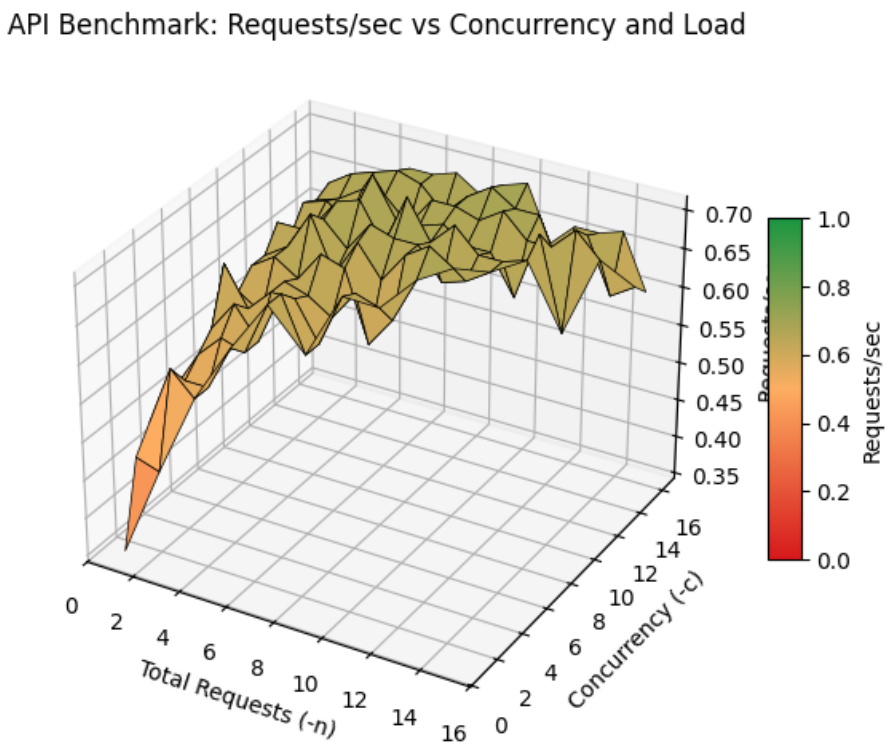
srovnatelně stejným zatížením od operačního systému Ubuntu 24.02. První měření proběhlo na implementaci v jazyce Python



Obrázek 4.2: Porovnání rychlosti čtení datasetu

Z grafu 4.2 můžeme vidět, že nízké počty celkových požadavků byly obslouženy decentně, ovšem s rostoucím sekvenčním zatížením výkon python serveru klesá. Lze spekulovat nad důvodem takového poklesu, ozvláště když je pokles vázaný na počet celkových požadavků a ne na počet souběžných požadavků. Jedním z možných vysvětlení je manipulace paměti při alokování a dealokování proměnných, které mohou negativně ovlinit následné požadavky. Problémy se správou paměti a *garbage collection* algoritmy je u intepretovaných jazyků znám již dlouho, a je to jedna z věcí, kterou je třeba zvážit při výběru platformy. Další možností, či pouze faktorem, je prodleva při manipulaci s vlákny, které Flask používá k obsluze požadavků. V některých případech je pohodlí vývoje na úkor menší výkonové ztráty přijatelná. V případě systému AITechDetect ovšem není, a proto bylo přistoupeno k implementaci v jiném, kompilovaném, programovacím jazyce.

Volbou pro novou implementaci byl jazyk Go. Tento jazyk skvěle pasuje do prostředí kontejnerů, kdy výstupní obrazy mají minimální velikosti, a zároveň je optimalizovaný na webové aplikace. Navíc nemá natolik komplikovanou syntaxi, aby přechod z jazyka Python byl pro zbytek vývojového týmu složitý. Byl tedy implementován koncový bod pro načtení datasetu, který využívá gorutiny a synchronizační primitiva, a bylo provedeno další měření.



Obrázek 4.3: Porovnání rychlosti čtení datasetu

Normalizace webového API

Původní webové API postrádalo konzistentní strukturu. CWE-1099 (Inconsistent Naming Conventions for Identifiers) upozorňuje na riziko, které s tím souvisí – konkrétně obtížnější lokalizaci chyb a slabin. Koncové body API byly nepředvídatelné a nebyly ani řádně dokumentované.

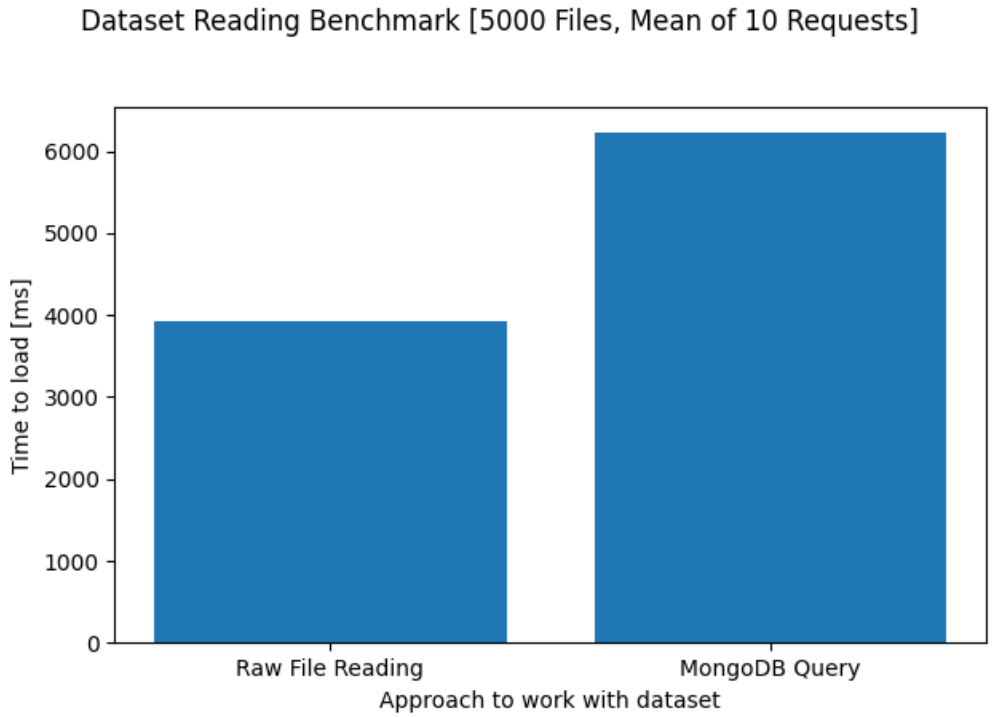
Řešením bylo co nejvíce normalizovat webové rozhraní podle standardu REST. Pro jednotlivé entity byly vytvořeny patřičné CRUD operace a byla zavedena architektura MVC s menšími úpravami. `Controller` zajišťuje obsluhu jednotlivých cest, `model` se stará o interakci s databází a místo tradičních pohledů jsou vytvořeny `services`, které poskytují obecné funkce, například připojení k databázi nebo validaci dat. Využití architektury MVC přivedlo konzistenci při vytváření nových koncových bodů, protože lze jednoznačně oddělit odpovědnosti jednotlivých částí.

Deklarativní validace vstupních dat

Generování dokumentace z anotací

4.3.2 Využití dokumentové databáze pro práci s JSON soubory

Jak bylo již řečeno v [[[[[Bakalářské práci, tady musíme dát proper odkaz]]]]], systém AITechDetect pracuje převážně nad soubory JSON. Každý dataset může mít od nižších desítek a po nižší desetitisíce takových souborů, a obzvláště na té větší spektra dochází k výrazném zpomalení při práci s datasety.



Obrázek 4.4: Porovnání rychlosti čtení datasetu

4.3.3 Předělání systému projektů

4.3.4 Přidání systému specifikací pro definici inspekce

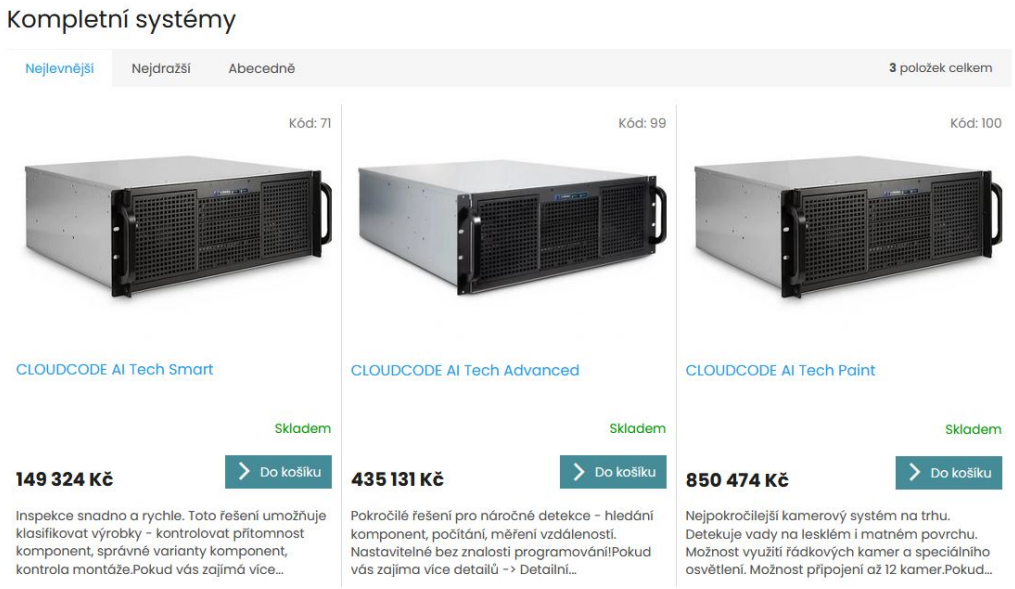
Lorem ipsum už něco máme z Hajdíku, ale je to hard coded.

4.3.5 Přidání systému pro správu toolů

- Seznam toolů a jejich verzí - V seznamu projektů vidět nasazené verze - Evaluační metriky z https://en.wikipedia.org/wiki/F-score

4.3.6 Přidání důkladné autentizace a autorizace

Jak bylo popsáno v analýze původního stavu řešení, aplikace nyní postrádá autentizaci a autorizaci, čímž se otevírají dveře k slabinám, které jsou popsané v CWE-284. Než vymýšlet vlastní řešení a řešit všechny bezpečnostní problémy s tím spojené, včetně auditů, je výhodnější využít již existující řešení, které je prověřené a aktivně používané. Z tohoto důvodu se pro autentizaci a autorizaci využil Keycloak. Ten je distribuován pod licencí Apache License 2.0, která umožňuje komerční užití bez licenčních poplatků a jiných právních omezení, které by jinak bránily jeho využití v komerční sféře.



Obrázek 4.5: Nasazení Keycloak kontejneru do systému AITechDetect

Keycloak umožňuje autentizaci pomocí protokolu OpenID, přičemž služby následně mohou uživatele autorizovat na základě dotazu, ve kterém přiloží informaci o chráněném zdroji a uživatelově tokenu. Keycloak pak rozhoduje, jestli má být uživatel autorizován či nikoliv. Služba má tak možnost vytvořit vlastní autorizační logiku, nebo ji delegovat na Keycloak.

Jední

4.3.7 Přidání procesu sestavení řešení

4.3.8 Předělání procesu nasazení

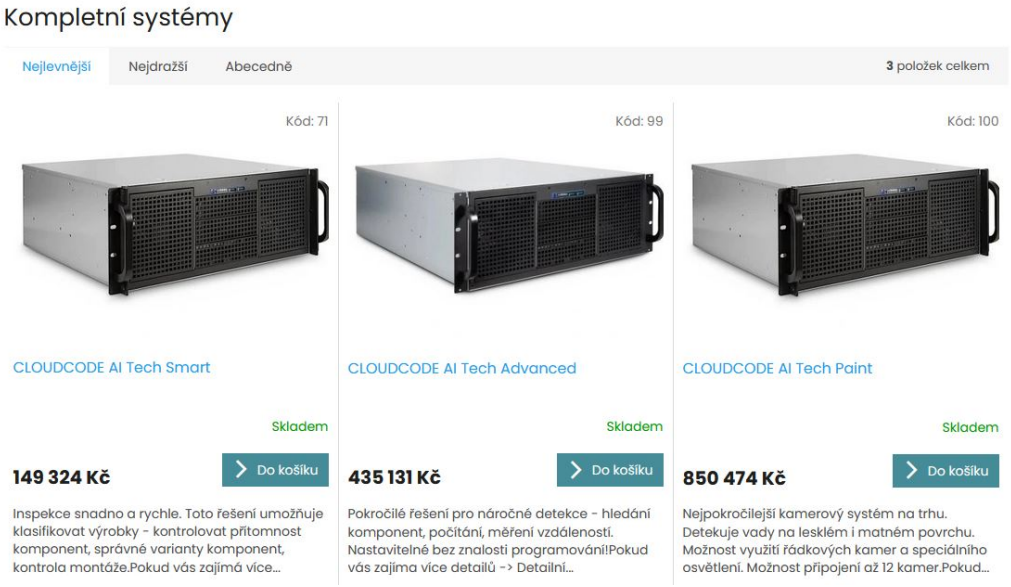
4.3.9 Přidání observability a monitoringu

4.3.10 Přidání licenčního systému

Jedním z požadavků společnosti CLOUDCODE bylo vytvořit systém licencování a zajistit, že software bude použit pouze oprávněnými uživateli

a v souladu s podmínkami licence. Tento systém musí řešit dva problémy - jak zabránit neoprávněné manipulaci se soubory poskytující informaci o licenci, a jak zabránit neoprávněné reprodukci.

AITechDetect je rozdělen do tří licencí, které se liší svými možnostmi a cenou. Licence jsou koncipované do hiearchie, kdy každá další úroveň obsahuje všechny funkce té předchozí. Cílem je poskytnout možnost levnější integrace výměnou za omezenější možnosti, které jsou pro danou problematiku dostačující. Na toto rozdělení funkcí je ale potřeba připravit i samotné řešení, a zabezpečit, že software bude odolný vůči účelné manipulaci zvenčí.



Obrázek 4.6: AITechDetect licence

Integrita a platnost licence je řešena pomocí kryptograficky podepsaného souboru, čímž se zabrání jeho manipulaci. Aby vše bylo bezpečné, byla využita asymetrická kryptografie. Licenční soubor je podepsán privátním klíčem zaměstnance CLOUDCODE a aplikace AITechDetect využívá veřejný klíč k ověření daného podpisu.

Samotná implementace licenčního systému byla koncipována jako webová služba, která implementuje rozhraní nad *GNU Privacy Guard* (GPG). Ten slouží ke správě klíčů a šifrování dat, a běžně se používá pro generování klíčů například pro SSH. GPG ovšem není jenom na vytvoření páru klíčů. Umožňuje také z privátního klíče generovat podklíče. Ty jsou využity jako další bezpečnostní opatření, ale tentokrát na straně samotné společnosti. Licenční systém totiž pro dané zaměstance vygeneruje unikátní podklíče, čímž je zajištěno, že původní privátní klíč nikdy nepřijde do styku se zaměstanci ani se zákazníky a je tak razantně nižší šance jeho kompromitace.

Posledním bezpečnostním opatřením bylo napojení licenčního systému na autorizační kontejner. Aby mohl zaměstnanec vytvářet licence, musí mít k tomu patřičná oprávnění definovaná v aplikaci Keycloak. Pokud tak není učiněno, licenční systém odmítne vystavit zaměstnanci jak podklíč, tak i podepsanou licenci. Tím je zajištěno, že k vytváření licencí má přístup pouze kontrolovaný počet zaměstnanců, kteří jsou tímto úkolem pověřeni a pro které případně plyne odpovědnost z nekalého počínání.

Podepsáním licenčního souboru je zajištěna integrita dat o zakoupené licenci, ovšem je potřeba také řešení ochránit před neoprávněnou reprodukcí. Počet možností zásadně snižuje fakt, že ověření licence musí kvůli bezpečnostním zásadám klientů fungovat offline. Nad tímto problémem byla vedena dlouhá úvaha. Prvotní myšlenka byla vytvořit otisky zařízení, ovšem řešení nasazováno v kontejnerech a tak nejde tak efektivně vytvářet otisky, nemluvě o situaci, kdy s dostatečným úsilím lze otisky zfalšovat. Další možností byly hardwarové klíče, které sice představují bezpečnější variantu, ale vyžadují fyzické umístění čipu do stanice nebo využití TPM. Tento způsob byl kvůli svému poměru náročnosti ku výsledku zamítnut.

Finálním řešením této kopírovací otázky byla změna úhlu pohledu. Než-li se snažit zamezit neoprávněné reprodukci, je z hlediska času i zdrojů lepší ji ošetřit právně. Pokud je zákazníkem zakoupena licence na AITechDetect, může sice software a licenci zkopírovat, ale bude muset čelit potenciálním právním důsledkům. Mezi ně může patřit například smluvní pokuta ve výši násobku ceny projektu. Na podobném principu fungují dohody o mlčenlivosti, které sdílení utajené informace nezastaví, ale potrestají odstrašující pokutou. Dalším faktorem je, že i při zatajení neoprávněné reprodukce se musí viník obejít bez podpory ze strany společnosti CLOUDCODE, protože žádáním o podporu by odhalil svůj přečin vůči smlouvě.

5 ZÁVĚR

Analýzy přiložené k textu práce nejsou nijak redigovány, protože neodpovídají nynějšímu skutečnému stavu.

BIBLIOGRAFIE

DANĚK, Kevin, 2024. *Rapidní vývoj algoritmů strojového vidění* [online]. 2024-06-11. [cit. 2026-03-02]. Dostupné z: https://dspace.tul.cz/handle/15240/175790.

ELASTIC OBSERVABILITY TEAM, 2024. *The 3 pillars of observability: Unified logs, metrics, and traces* [online]. 2024-10-10. [cit. 2025-08-09]. Dostupné z: https://www.elastic.co/blog/3-pillars-of-observability.

FORRESTKNIGHT, 2025. *Everything You NEED to Know About TECH DEBT* [online]. 2025-07-24. [cit. 2025-08-10]. Dostupné z: https://www.youtube.com/watch?v=ukgmp6uxQJc.

FOWLER, Martin, 2009. *Technical Debt Quadrant* [online]. 2009-10-14. [cit. 2025-08-10]. Dostupné z: https://martinfowler.com/bliki/TechnicalDebtQuadrant.html.

MARTIN, Robert C.; COPLIEN, James O., 2009. *Clean code: a handbook of agile software craftsmanship*. Prentice Hall. ISBN 9780132350884.

MICHIGAN TECHNOLOGICAL UNIVERSITY, 2025. *What is software engineering?* [online]. 2025-04-09. [cit. 2026-03-03]. Dostupné z: https://www.mtu.edu/cs/undergraduate/software/what/.

SHVETS, Alexander, 2018. *Technical Debt* [online]. 2018-02-08. [cit. 2026-03-03]. Dostupné z: https://refactoring.guru/refactoring/technical-debt.

TEAM ASANA, 2025. *What is technical debt? How to pay it off (with examples)* [online]. 2025-06-25. [cit. 2025-08-10]. Dostupné z: https://asana.com/resources/technical-debt.